Advance Seminar Series Midterm Output

Members:

Klarence A. De Gracia
Syclops S. Lariosa
Cyrel Jave M. Morales

Overview

For this Advance Seminar requirement, we prepared the EnerSight repository so that AI coding assistants can work on it safely and consistently. EnerSight is a Building-Specific Energy Consumption Mapping and Optimization System using GIS, built with a FastAPI + PostgreSQL backend, a React (Vite) + Tailwind + Leaflet frontend, and OCR-based meter reading. Our work covered four tasks: opening a dedicated working branch, writing guidance files for AI tools, creating two custom skills for our own codebase, and installing a ready-made skill from an outside provider. Doing these gave us hands-on practice in setting up a project so that AI-assisted development stays organized and follows our rules.

Repository: https://github.com/Ajjime/EnerSight

Creating new branch

Our group created a new Git branch named `day2/login-page-redesign` in the EnerSight repository. We used it as the working branch for this activity so our changes stayed separate from `main`; `AGENTS.md` also forbids committing directly to `main`. The branch holds the guidance files and custom skills, plus one small, scoped feature the AI assistant implemented using them: the login page redesign with entrance animations and micro-interactions. Working on a branch kept each change small, reviewable, and easy to recover from.

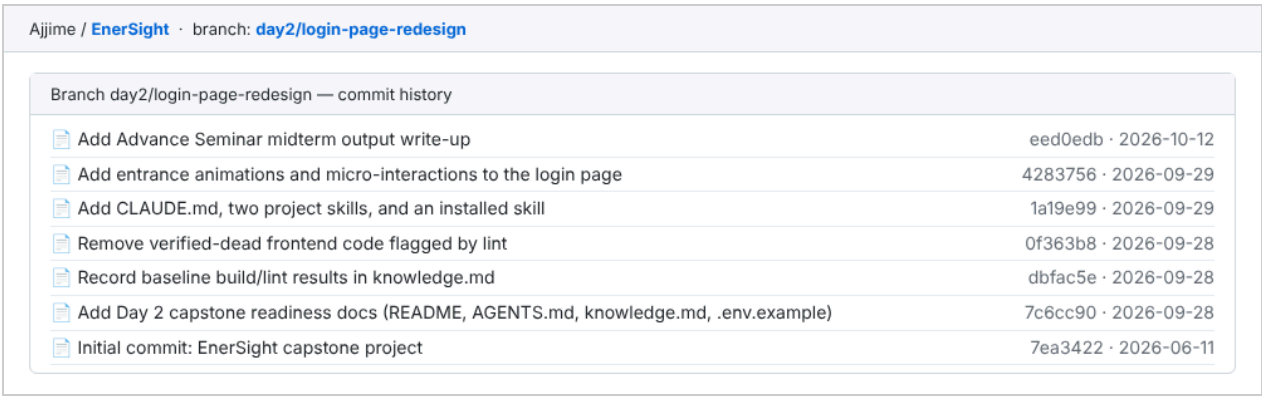


Figure 1. Commit history of the day2/login-page-redesign branch in the EnerSight repository.

Initialize/Create a knowledge.md/AGENTS.md/CLAUDE.md

Our group created three guidance files at the root of the EnerSight repository so AI coding assistants understand the project before changing code:

- `AGENTS.md` (working rules): what to read first, the rule to use a feature branch, folder boundaries (backend routers each own one resource; frontend pages vs. components vs. hooks), files that must never be committed (`.env`, `venv/`, `node_modules/`, uploads), the required checks before a task is done (`py_compile`, `npm run lint`, `npm run build`, manual browser check), and known constraints such as the hardcoded JWT `SECRET_KEY`.
- `knowledge.md` (verified facts): architecture map, data model (`User`, `Building`, `Meter`, `ConsumptionRecord`, `EnergyReport`, `AccessLog`), the API surface table, deliberate constraints, and a recorded lint/build baseline with the pre-existing lint findings explained.
- `CLAUDE.md` (Claude entry point): imports `AGENTS.md` and `knowledge.md`, lists the project skills and when to invoke them, and states that there is no automated test suite, so "verify" means running the checks and exercising the change manually.

Together these keep the AI tools within the existing project structure and avoid unnecessary modifications.

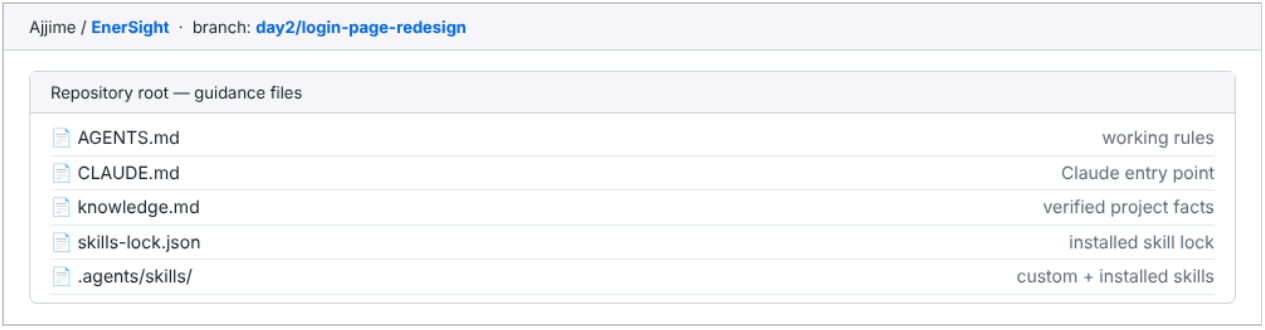


Figure 2. Project guidance files in the repository root.

Ajjime / **EnerSight** · branch: **day2/login-page-redesign**

AGENTS.md

AGENTS.md

Working rules for any AI agent (or contributor) making changes in this repository.

Before making changes

- Read `knowledge.md` first for architecture and contracts.
- Create a feature branch. Do not commit directly to `main`.
- Keep changes small and scoped to one feature or refactor at a time.

Boundaries

- `enersight-backend/app/` — FastAPI backend. Route modules under `app/routes/` each own one resource (buildings, meters, readings, users, auth, ocr, map, reports, analytics). Don't cross-edit unrelated routers for a single task.
- `enersight-frontend/src/pages/` — one page per screen. `src/components/` holds shared UI. `src/hooks/` holds shared stateful logic (e.g. `useAutoRefresh`).
- Never commit `.env`, `venv/`, `node_modules/`, or anything under `uploads/` / `meter_photos/` (already covered by `.gitignore`).
- Never hardcode secrets (API keys, DB passwords) in source files — use `.env` (see `enersight-backend/.env.example`).

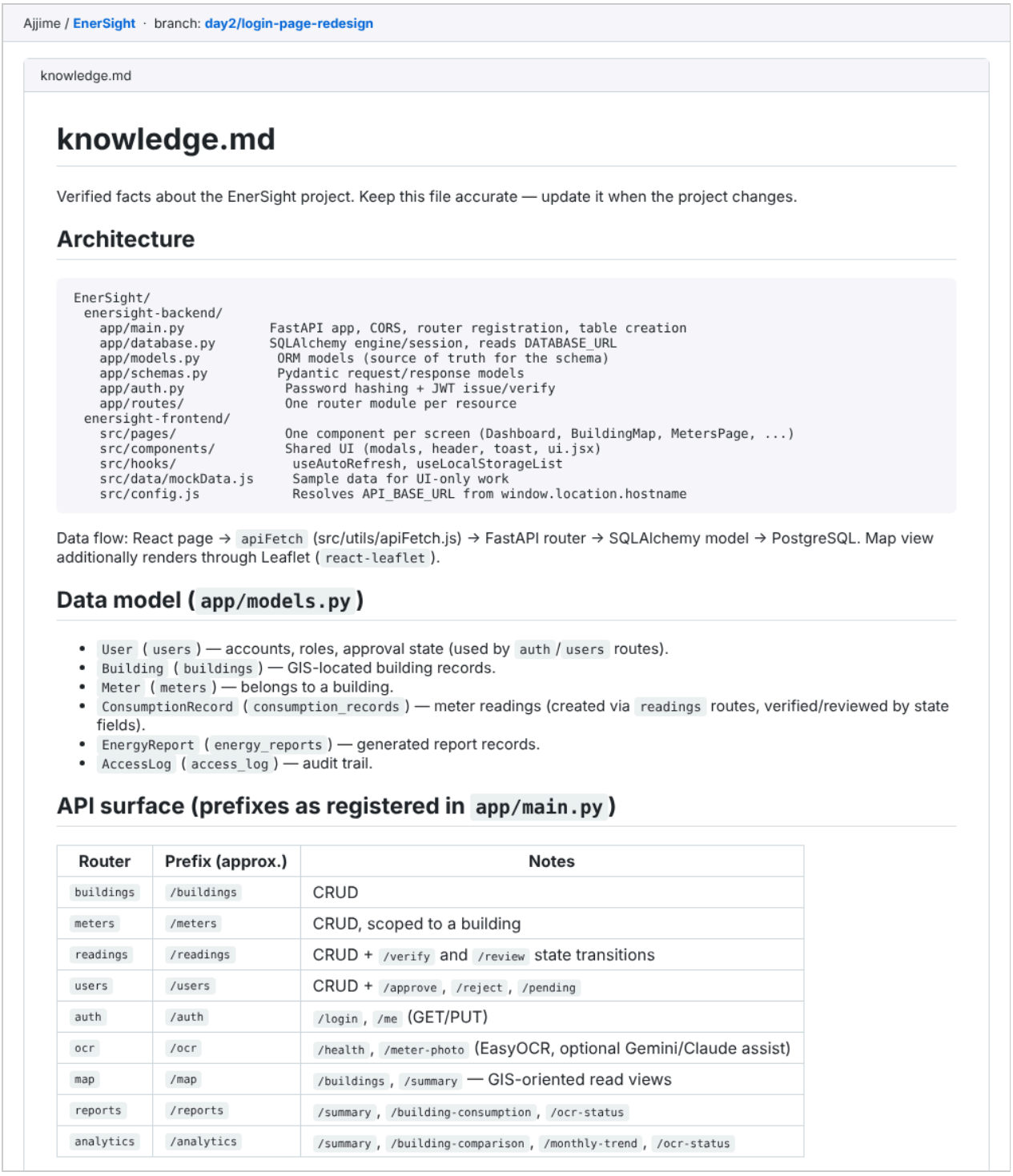
Required checks before calling a task done

- Backend: `python -m py_compile app/main.py` (or run the server and hit the touched endpoint).
- Frontend: `npm run lint` and `npm run build` in `enersight-frontend/`.
- Manually exercise the touched flow in the browser — there is no automated test suite yet, so this is the only verification available.
- Report which checks were run and their result; don't claim a change works without running it.

Known constraints (see knowledge.md for detail)

- `DATABASE_URL` (PostgreSQL) is required — the backend will not start without it.
- `SECRET_KEY` for JWT auth is currently hardcoded in `app/auth.py` — flag this if touching auth, don't silently "fix" it without asking (it's a known pre-existing issue, not in scope unless asked).
- OCR (`app/routes/ocr.py`) can run locally via EasyOCR or defer to Gemini/Anthropic if `GEMINI_API_KEY` / `ANTHROPIC_API_KEY` are set.

Figure 3. AGENTS.md containing the project's working rules.



Ajjime / EnerSight · branch: day2/login-page-redesign

knowledge.md

knowledge.md

Verified facts about the EnerSight project. Keep this file accurate — update it when the project changes.

Architecture

```
EnerSight/
  enersight-backend/
    app/main.py           FastAPI app, CORS, router registration, table creation
    app/database.py       SQLAlchemy engine/session, reads DATABASE_URL
    app/models.py          ORM models (source of truth for the schema)
    app/schemas.py         Pydantic request/response models
    app/auth.py             Password hashing + JWT issue/verify
    app/routes/             One router module per resource
  enersight-frontend/
    src/pages/              One component per screen (Dashboard, BuildingMap, MetersPage, ...)
    src/components/         Shared UI (modals, header, toast, ui.jsx)
    src/hooks/               useAutoRefresh, useLocalStorageList
    src/data/mockData.js     Sample data for UI-only work
    src/config.js            Resolves API_BASE_URL from window.location.hostname
```

Data flow: React page → `apiFetch` (src/utils/apiFetch.js) → FastAPI router → SQLAlchemy model → PostgreSQL. Map view additionally renders through Leaflet (`react-leaflet`).

Data model (`app/models.py`)

- `User` (`users`) — accounts, roles, approval state (used by `auth` / `users` routes).
- `Building` (`buildings`) — GIS-located building records.
- `Meter` (`meters`) — belongs to a building.
- `ConsumptionRecord` (`consumption_records`) — meter readings (created via `readings` routes, verified/reviewed by state fields).
- `EnergyReport` (`energy_reports`) — generated report records.
- `AccessLog` (`access_log`) — audit trail.

API surface (prefixes as registered in `app/main.py`)

Router	Prefix (approx.)	Notes
`buildings`	`/buildings`	CRUD
`meters`	`/meters`	CRUD, scoped to a building
`readings`	`/readings`	CRUD + `/verify` and `/review` state transitions
`users`	`/users`	CRUD + `/approve`, `/reject`, `/pending`
`auth`	`/auth`	`/login`, `/me` (GET/PUT)
`ocr`	`/ocr`	`/health`, `/meter-photo` (EasyOCR, optional Gemini/Claude assist)
`map`	`/map`	`/buildings`, `/summary` — GIS-oriented read views
`reports`	`/reports`	`/summary`, `/building-consumption`, `/ocr-status`
`analytics`	`/analytics`	`/summary`, `/building-comparison`, `/monthly-trend`, `/ocr-status`

Figure 4. knowledge.md containing the architecture and technical information (top section).

Create two (2) custom skills

We created two project-specific skills under `.agents/skills/`, each covering a different layer of the project.

Skill 1: `enersight-frontend-page`

Guides AI assistants when planning, implementing, or reviewing a change to a React page or component. It tells the assistant to read `knowledge.md` first and reuse the existing layout shells (`AuthShell`, `PageHeader`), `lucide-react` icons, the keyframes in `src/index.css`, and inline Tailwind classes. It protects the `apiFetch` contract and the auth `localStorage` keys, warns that ESLint falsely flags `icon: Icon` props as unused, and requires `npm run lint`, `npm run build`, and a manual browser check.

Skill 2: `enersight-backend-route`

Guides AI assistants when changing a FastAPI route, Pydantic schema, or SQLAlchemy model. It has the assistant trace router, `schemas.py`, `models.py`, and `database.py`, follow the `Depends(...)` auth convention, avoid SQLite assumptions (PostgreSQL only), edit one router at a time, and verify with `py_compile` and a real request to the touched endpoint.

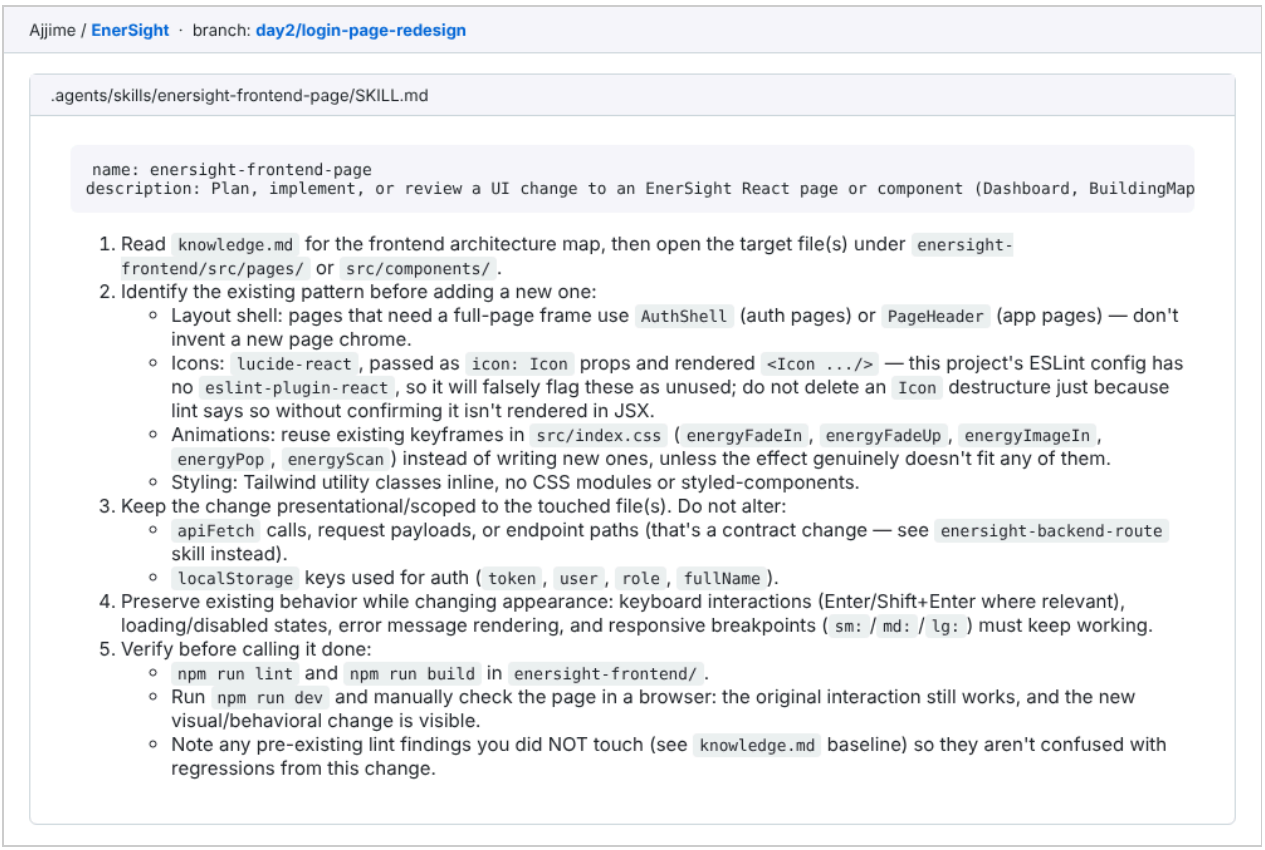


Figure 5. SKILL.md of the enersight-frontend-page custom skill.

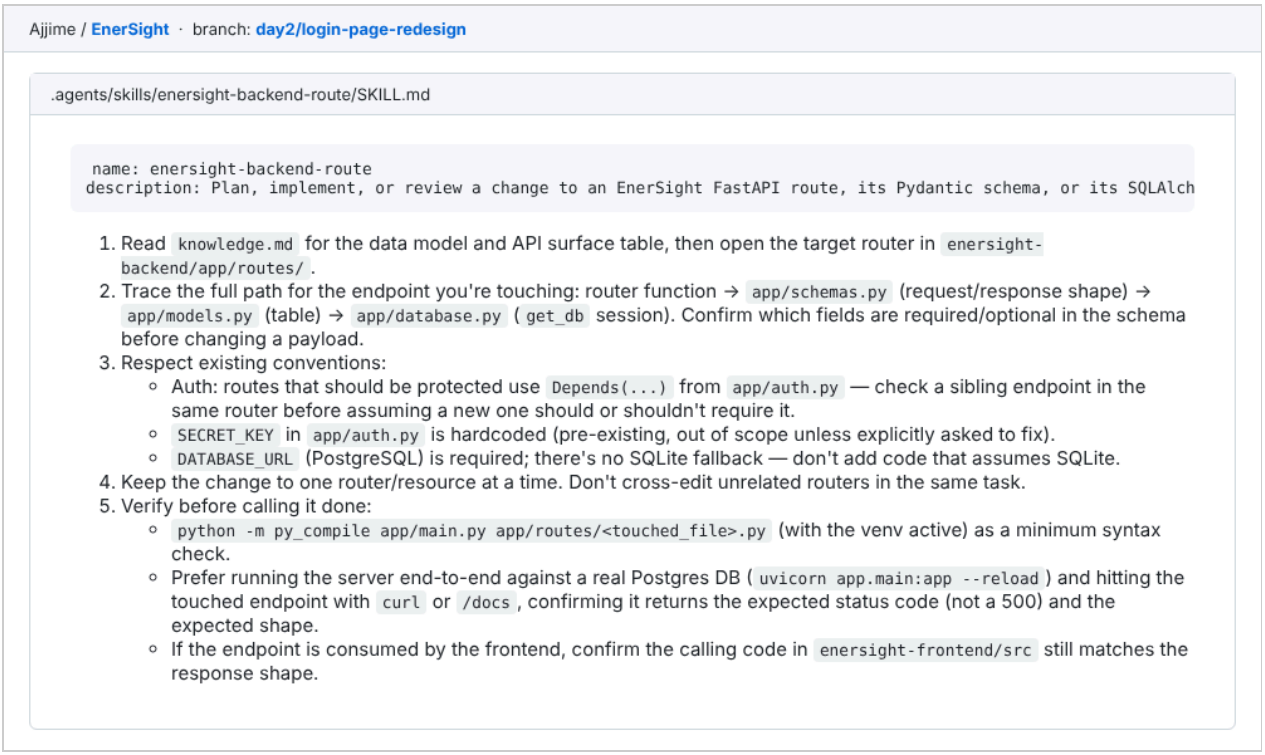


Figure 6. SKILL.md of the enersight-backend-route custom skill.

Download/Install skills from skills provider

We installed the `vercel-react-best-practices` skill from the `vercel-labs/agent-skills` GitHub repository into `.agents/skills/vercel-react-best-practices/` (a `SKILL.md`, an `AGENTS.md`, and a `rules/` folder of individual React/performance rules). The install is recorded in `skills-lock.json` with its source and content hash so it can be reproduced. We use it as a reference for frontend work such as bundle size, re-renders, and data fetching, alongside our own skills and guidance files.

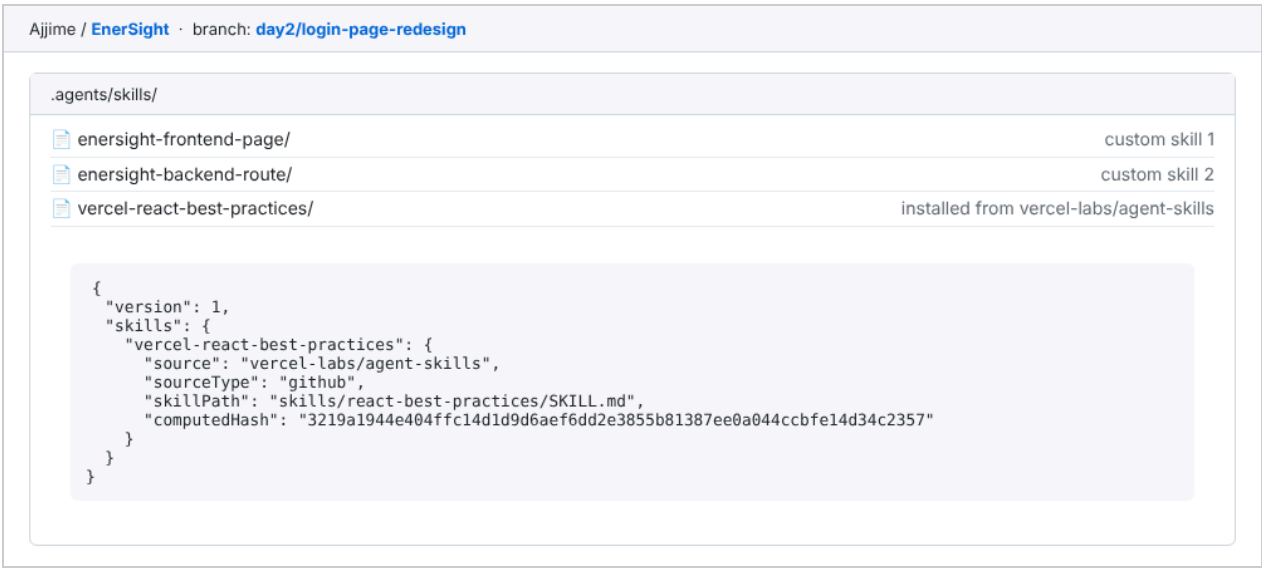


Figure 7. Skills folder and skills-lock.json recording the installed skill.

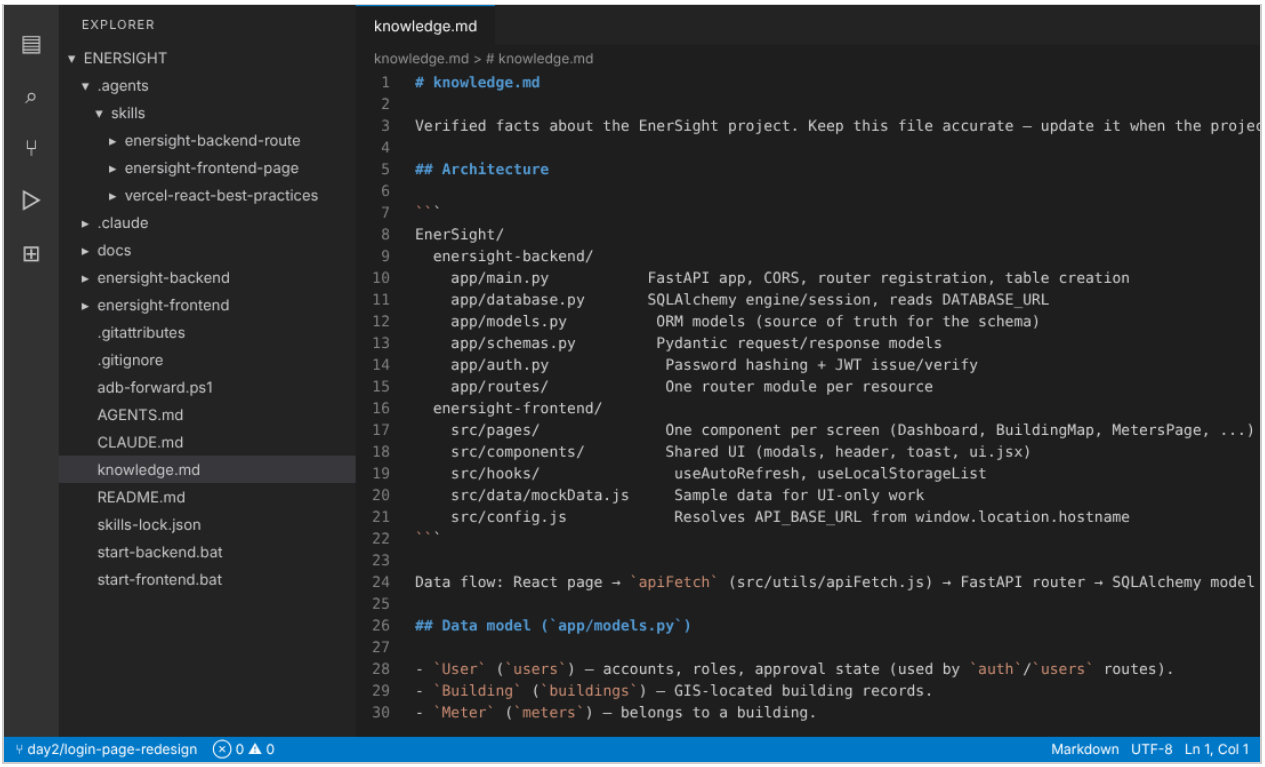


Figure 8. Project guidance files and skills in Visual Studio Code (Explorer on the left, knowledge.md open).

Conclusion

Through the four processes of the Advance Seminar activity, our group practiced organizing a GitHub branch, preparing project guidance files, creating custom skills, and installing a skill from a provider. We learned that clear documentation and reusable guidelines help AI coding assistants follow a project's structure and rules, and that keeping each task to one small change on its own branch makes AI-assisted work easier to review and verify.